

■ Functional Programming

Python — Functions, Lambda, Map, Filter, Reduce & Decorators

#	Topic
1	Pure Functions
2	Higher-Order Functions
3	Defining Functions (def)
4	Code Reusability
5	Prime Number Function — Example
6	*args & **kwargs
7	Lambda Functions
8	map()
9	filter()
10	reduce()
11	Decorators

1 · Pure Functions

- Always produce the **same output** for the same input
- Have **no side effects** — don't modify external state
- Easier to test, debug, and reason about

Characteristic	Meaning
Deterministic	Same input always → same output
No side effects	Doesn't modify variables outside itself
Referentially transparent	Can replace call with its return value

2 · Higher-Order Functions

A **Higher-Order Function** is one that does at least one of:

- **Returns** another function as its result
- **Takes** a function as a parameter / argument

■ *map(), filter(), reduce(), and decorators are all higher-order functions.*

3 · Defining Functions (def)

```
Python
1 def function_name(parameters):
2     # function body
3     return value
```

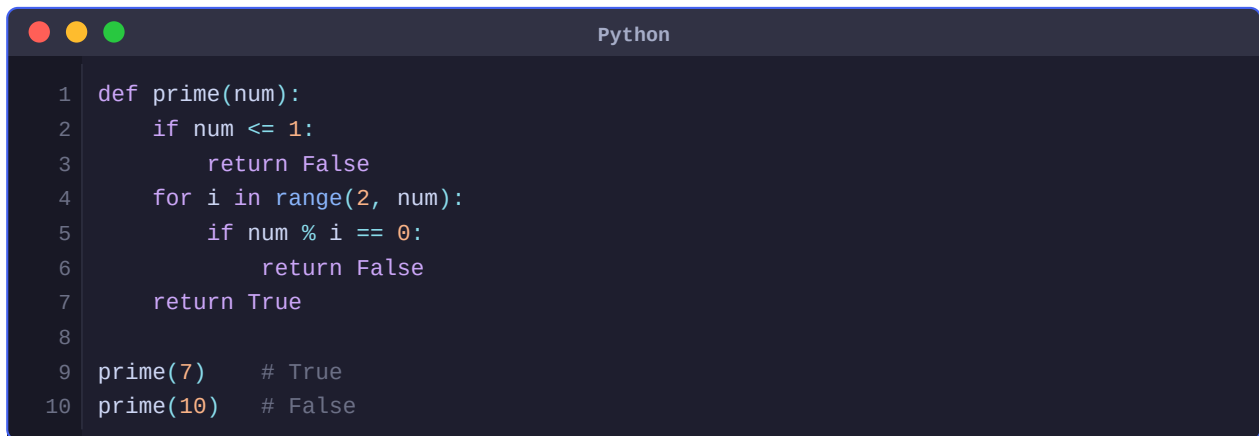
Part	Description
def	Keyword that starts a function definition
function_name	Name you give to the function (snake_case by convention)
parameters	Input values the function accepts (optional)
return	Sends a value back to the caller (optional — returns None if omitted)

4 · Code Reusability

- Write once, use many times — avoids repeating the same logic
- Makes code shorter, cleaner, and easier to maintain
- Bugs only need to be fixed in one place
- Functions can be imported and shared across files/modules

5 · Example — Prime Number Function

A number is **prime** if it is greater than 1 and divisible only by 1 and itself.

A screenshot of a Python IDE window with a dark background. The window has three colored window control buttons (red, yellow, green) in the top-left corner and the title "Python" in the top-right corner. The code is written in a light-colored font. It defines a function `prime(num)` that checks if a number is prime. The function returns `False` for numbers less than or equal to 1, and for numbers greater than 1, it checks for divisibility by integers from 2 to `num-1`. If any divisor is found, it returns `False`; otherwise, it returns `True`. Below the function definition, there are two test cases: `prime(7)` which is commented as `# True`, and `prime(10)` which is commented as `# False`.

```
1 def prime(num):  
2     if num <= 1:  
3         return False  
4     for i in range(2, num):  
5         if num % i == 0:  
6             return False  
7     return True  
8  
9 prime(7)      # True  
10 prime(10)    # False
```

■ *Optimisation: use `range(2, int(num**0.5)+1)` instead of `range(2, num)` — you only need to check up to the square root.*

6 · *args & **kwargs

Parameter	Meaning	Receives
*args	Variable number of positional arguments	Tuple of values
**kwargs	Variable number of keyword arguments	Dict of key=value pairs

*args example

```
Python
1 def addition(*args):
2     result = sum(args)
3     return result
4
5 addition(2, 3, 4)  # 9 — any number of arguments
```

**kwargs example

```
Python
1 def display_info(**kwargs):
2     for key, value in kwargs.items():
3         print(f'{key}: {value}')
4
5 display_info(name='Alice', age=30)
6 # name: Alice
7 # age: 30
```

■ Tip: You can combine both: `def func(*args, **kwargs)` — `args` must come before `kwargs`.

7 · Lambda Functions

A **lambda** is an anonymous (nameless), single-expression function.

```
Syntax
1 lambda argument : expression
```

```
Python
1 # Regular function
2 def add(a, b):
3     return a + b
4
5 # Equivalent lambda
6 add = lambda a, b: a + b
7
8 add(7, 9)  # 16
```

Feature	def	lambda
---------	-----	--------

Name	Has a name	Anonymous (no name)
Body	Multiple statements allowed	Single expression only
return	Explicit return	Implicit (expression value is returned)
Use case	General-purpose functions	Short throwaway / inline functions

8 · map()

map() applies a function to **every element** of an iterable and returns a map object.

```
Syntax
1 map(function, iterable)
```

```
Python
1 list1 = [1, 2, 3, 4]
2
3 def square(n):
4     return n ** 2
5
6 result = list(map(square, list1))
7 # [1, 4, 9, 16]
8
9 # Same thing with lambda (more concise)
10 result = list(map(lambda n: n ** 2, list1))
11 # [1, 4, 9, 16]
```

■ *map() returns a lazy map object — wrap it in list() to see the results.*

9 · filter()

filter() keeps only elements for which the function returns **True**.

```
Syntax
1 filter(function, iterable)
```

```
Python
1 nums = [1, 2, 3, 4, 5, 6, 7]
2
3 evens = filter(lambda x: x % 2 == 0, nums)
4 list(evens) # [2, 4, 6]
```

Function	Acts on	Returns
map()	Every element	Transformed value for each element
filter()	Every element	Only elements where function returns True

10 · reduce()

reduce() cumulatively applies a function to reduce an iterable to a **single value**. It must be imported from `functools`.

```
Python
1 from functools import reduce
2
3 nums = [1, 2, 3, 4, 5, 6, 7]
4 result = reduce(lambda x, y: x + y, nums)
```

How `reduce()` works step-by-step

Step	Calculation	Running Total
1	1 + 2	3
2	3 + 4	7
3	7 + 5	12
4	12 + 6	18
5	18 + 7	25 ← final result

■ *`reduce()` processes pairs left-to-right, accumulating a single result. The list `[1,2,3,4,5,6,7]` sums to 25.*

11 · Decorators

A **decorator** is a higher-order function that **wraps another function** to extend its behaviour — without changing its source code.

Concept	Description
@decorator_name	Syntax sugar — equivalent to: func = decorator(func)
wrapper()	Inner function that runs before/after the original function
return wrapper	The decorator returns the wrapper to replace the original

Decorator — Example

```
Python
1 def decorator(func):
2     def wrapper():
3         print('start')
4         func()
5         print('end')
6     return wrapper
7
8 @decorator
9 def say_hello():
10     print('Hello!')
11
12 say_hello()
13 # Output:
14 # start
15 # Hello!
16 # end
```

The @decorator syntax above is equivalent to writing:

```
Python
1 def say_hello():
2     print('Hello!')
3
4 say_hello = decorator(say_hello) # manual decoration
```

■ *Decorators are used heavily in frameworks like Flask (@app.route), Django, and for logging, timing, authentication, and caching.*

Quick Reference — Functional Tools

Tool	Purpose	Import needed?
def	Define a named function	No
return	Send a value back to caller	No

lambda	Inline anonymous function	No
*args	Accept any number of positional args	No
**kwargs	Accept any number of keyword args	No
map()	Apply function to every element	No
filter()	Keep elements where function returns True	No
reduce()	Collapse iterable to single value	from functools import reduce
@decorator	Wrap a function to extend its behaviour	No

```

Python

1 # All three together – functional style pipeline
2 from functools import reduce
3
4 nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
5
6 result = reduce(
7     lambda x, y: x + y,
8     map(lambda n: n ** 2,
9         filter(lambda n: n % 2 == 0, nums))
10 ) # sum of squares of even numbers: 4+16+36+64+100 = 220

```

■ *Tip: Chaining map → filter → reduce is a classic functional programming pattern — readable and avoids intermediate variables.*